# Initial Plan: Architectural Limits of Interpreter Design

Author: **Darsh Chanduka**

Supervisor: **Ramalakshmi Vaidhiyanathan**

## Project Description

### Context and Motivation

Instruction set emulation is foundational to modern systems engineering. The technology is widely used in hardware design verification, in architectural simulators for academic research, and in legacy hardware emulation for digital preservation. In these domains, the primary goal is to reproduce the fetch–decode–execute behaviour of a target architecture on a host machine.

Real-time instruction set emulation [1] is usually implemented using one of two primary strategies: interpretation [2] or just-in-time (JIT) compilation [3]. Both approaches solve the same core problem—mapping target instructions onto host instructions—but they have different strengths and weaknesses. Interpreters decode and execute instructions at runtime, whereas JIT compilers translate blocks of target code into host code at runtime.

Traditionally, these approaches are viewed as a binary choice: interpreters are simple but slow, while JIT compilers are complex but fast. This view, however, does not fully reflect the constraints of modern systems. In many industrial applications, the complexity of JIT compilation is a liability. In safety-critical systems, dynamic code generation is often prohibited due to security risks and verification difficulties. In hardware modelling, the portability and low memory footprint of an interpreter are often more valuable than raw execution speed.

### Problem Definition and Aim

This thesis examines interpreter performance, complexity and costs in relation to JIT compilers. Rather than relying on informal comparisons, it provides a structured empirical evaluation of these trade-offs to determine when a JIT transition is justified. The central question motivating this work is:

**Can a sophisticated interpreter approach JIT-level performance without incurring JIT-level complexity?**

Crucially, the interpreters' performance does not need to match or surpass the JIT compiler. It just needs to reach an adequate level, after which performance is no longer the central issue and focus shifts from performance to other factors, such as maintainability and extensibility.

[1] Smith and Nair (2005)
[2] Mihocka (2008)
[3] Sharp and Martin (2000)

### Proposed Approach

Rather than directly comparing interpreters against fully optimised JIT compilers, this project will investigate interpreter evolution as a continuum. It will examine how architectural changes to instruction dispatch and execution models affect performance, implementation complexity, and maintainability, and how these designs converge toward JIT-like structures before native code generation is introduced.

Importantly, the distinction between JIT compilation and Interpretation is defined by the use of **native code generation**. JIT compilation is treated as an architectural boundary beyond which implementation complexity increases sharply.

The MIPS instruction set architecture is used as a concrete case study due to its simplicity, well-defined semantics, and extensive documentation. The focus of the investigation is not on MIPS-specific optimisations, but on **general interpreter design patterns** that are applicable across instruction sets and virtual machine implementations.

So far, three interpreters have been envisioned: a canonical switch-based one, a computed-goto-based one [4], and a decoupled cache-aware one [5]. Performance will be assessed using controlled benchmarks and hardware performance counters, while complexity will be examined through static code metrics and architectural analysis.

Justification and Scope

The project presents a substantial technical and analytical challenge in line with the project module. It involves the design, implementation, and evaluation of multiple interpreter architectures, requiring systems-level programming, experimental measurement, and critical architectural reasoning.

The scope is deliberately constrained to prioritise analytical clarity and feasibility. Advanced techniques such as speculative optimisation, aggressive register allocation, and full native code generation are excluded, as they would obscure the architectural trade-offs under investigation. This allows the project to focus on generalisable insights rather than maximal performance.

## Objectives

### Interpreters

Design and implement several interpreters for the MIPS I Instruction Set Architecture (ISA). Each variant will represent a different level of architectural complexity. Currently, three interpreters have been planned: a canonical switch-based interpreter, a computed-goto-based interpreter, and a decoupled, cache-aware

[4] Bendersky (2026)
[5] "TCG Emulation — QEMU Documentation" (2022)

interpreter.

***Risk***: There is an inherent difficulty in creating each interpreter, and creating meaningfully different architectures for each interpreter is even more difficult.

***Mitigation***: Experience. This risk is heavily mitigated by my personal experience; I have read a large amount of literature surrounding Virtual Machine design, Interpreter methods, Systems Programming and Instruction Set Architectures. The effect of this can already be seen; selecting the MIPS I ISA simplifies the problem already. It is a comparatively simple architecture, with few instructions and fewer quirks to trip up on.

### Data gathering and Analysis

Define, Gather, and Analyse performance, complexity and cost metrics. Use a mixture of Quantitative and Qualitative measures, ensuring that the causes and quantities can verify each other. Ensure benchmarks are unbiased, not favouring specific workloads or patterns, allowing for a fair and insightful analysis.

***Risk***: There is an inherent difficulty in defining Complexity and Cost in a way that is objectively measurable. Quantitative metrics (like SLOC or Cyclomatic Complexity) may fail to capture the "Cognitive Load" of an architecture, while performance data can be highly sensitive to benchmark properties.

***Mitigation***: A dedicated phase at the project's outset will be used to establish standardised benchmarks and quantitative complexity metrics before any data gathering begins. By formalising these criteria early, the study ensures that "complexity" and "performance" are measured against a consistent academic yardstick, preventing mid-project bias and ensuring that later findings can be directly and meaningfully compared to established studies in the field.

## Feasibility

This project is a software-based investigation and does not involve human participants, personal data, or user studies. As such, no ethical approval is required.

There are no anticipated legal or intellectual property issues, as all code will be original and developed specifically for the project. Any third-party tools used for measurement or analysis will be open-source and appropriately cited.

The project does not require specialised hardware. Performance measurements will be conducted on standard development machines.

***Risk***: Incomplete Implementation of All Interpreter Variants. A potential risk to the feasibility of this study is the possibility that not all planned interpreter implementations can be completed within the available timeframe. In particular, the most sophisticated interpreter design involves substantial engineering effort and may prove difficult to fully implement, debug, or stabilise.

***Mitigation***: The study is structured around multiple interpreter variants of increasing sophistication. Should one implementation prove infeasible, the remaining interpreters will still enable meaningful comparative analysis across execution strategies. In such a case, the scope of the study would be adjusted to focus on the completed implementations, while explicitly documenting the limitations and implications of the reduced comparison set.

## Work Plan

### Research and Experimental Design (Weeks 1–3)

This phase is the foundation of the project. Relevant work on interpreter design, instruction dispatch methods, caching, and JIT compilation will be reviewed to guide the project direction. Benchmark programs, test cases, and evaluation metrics will be defined.

At the same time, supporting components will be developed. These include the memory system, binary loaders, and a small SDK for building benchmarks. By the end of this phase, draft versions of the background and methodology sections of the report will be completed.

**Deliverables & Milestones**

- Benchmark workloads, test programs, loaders, memory design, and test harnesses
- Defined performance, complexity, and practicality metrics
- Draft background and methodology sections

### Switch-Based Interpreter (Weeks 4–5)

In this phase, the first interpreter will be implemented. It will support a core subset of the MIPS instruction set and use a switch-based dispatch method. The focus will be on correctness, clear structure, and stable behaviour. This interpreter will serve as a baseline for later comparisons.

After validation, a draft report section describing the design and early performance results will be written.

**Deliverables & Milestones**

- Switch-based interpreter implementation
- Validation and correctness test results
- Draft report section for this interpreter

### Computed-Goto-Based Interpreter (Week 6)

This phase implements a second interpreter using a computed-goto dispatch method. The main task is to replace the dispatch mechanism while reusing the existing instruction set and support code. Performance differences and design changes will be analysed.

A draft report section describing the implementation and results will be completed.

**Deliverables & Milestones**

- Computed-goto-based interpreter implementation
- Initial performance measurements
- Draft report section for this interpreter

### Decoupled Cache-Aware Interpreter (Weeks 7–9)

This phase focuses on the most complex interpreter design. The interpreter separates instruction decoding and execution into different threads. This requires careful thread coordination and attention to cache and memory behaviour. All earlier techniques will be reused where possible.

Even if only a partial implementation is completed, a detailed design and analysis will be written.

**Deliverables & Milestones**

- Design documentation for the decoupled interpreter
- Prototype or partial implementation
- Draft report section for this interpreter

### Metric Collection and Analysis (Weeks 10–11)

During this phase, performance and complexity data will be collected for all interpreter versions. The results will be compared to highlight the trade-offs between simpler and more complex designs. The results and analysis sections of the report will be drafted.

**Deliverables & Milestones**

- Benchmark and performance data
- Complexity and static analysis results
- Comparative analysis across interpreter designs
- Draft results and analysis sections

### Final Report and Submission (Week 12)

This phase will bring together the report sections into a single document. Results will be refined, conclusions finalised, and supporting materials prepared for submission. The focus will be on clarity and presentation rather than new writing.

**Deliverables & Milestones**

- Final dissertation
- Final figures, tables, and data
- Documented codebase and appendices

## References


Bendersky, Eli. 2026. “Computed Goto for Efficient Dispatch Tables - Eli Bendersky’s Website.” Thegreenplace.net. https://eli.thegreenplace.net/2012/07/12/computed-goto-for-efficient-dispatch-tables.

Mihocka, Darek. 2008. “NO EXECUTE! September 3, 2008.” Emulators.com. http://www.emulators.com/docs/nx25_nostradamus.htm.

Sharp, David, and Graham Martin. 2000. “A Dynamically Recompiling ARM Emulator.” https://www.davidsharp.com/tarmac/tarmacreport.pdf.

Smith, James E, and Ravi Nair. 2005. *Virtual Machines : Versatile Platforms for Systems and Processes*. Morgan Kaufmann Publishers.

“TCG Emulation — QEMU Documentation.” 2022. Weilnetz.de. https://qemu.weilnetz.de/doc/7.2/devel/index-tcg.html.